

Inductive Biases, Diverse Ensembles and Calibrated Confidence

MIT-PSFC Machine Learning Working Group // 28-05-2025

Yoeri Poels | yoeri.poels@epfl.ch

Machine Learning!

- Learn function f that minimizes objective $\mathcal{L}$ using dataset $\mathcal{D} = \{x_i\}_{i=1}^N$
 - Often paired data $\{(x_i, y_i)\}_{i=1}^N$, $(x_i, y_i) \sim P_{X,Y}$. Learn mapping $f : X \rightarrow Y$

- For parametrized family of functions f_θ , minimize the empirical risk:

$$\theta^* = \arg \min_{\theta \in \Theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_\theta(x_i), y_i)$$

Inductive Biases

$$\theta^* = \arg \min_{\theta \in \Theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\theta}(x_i), y_i)$$

- Many choices that inform what we find
 - What is the functional family (architecture)?
 - What is the loss?
 - Regularization?
 - How do we find our parameters?

Inductive Biases: model

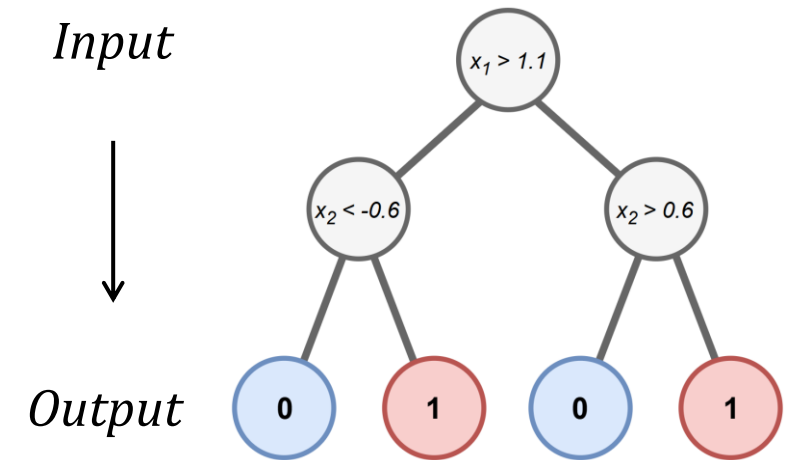
Example: type of model / architecture, f_θ

Decision tree:

- Start at root node
- Check a condition on input $\mathbf{x}$
- Move left/right depending on condition
- Repeat until reaching a leaf

(Consider parameters θ as the tree splits / leaf values)

- We partition the data in feature-aligned regions



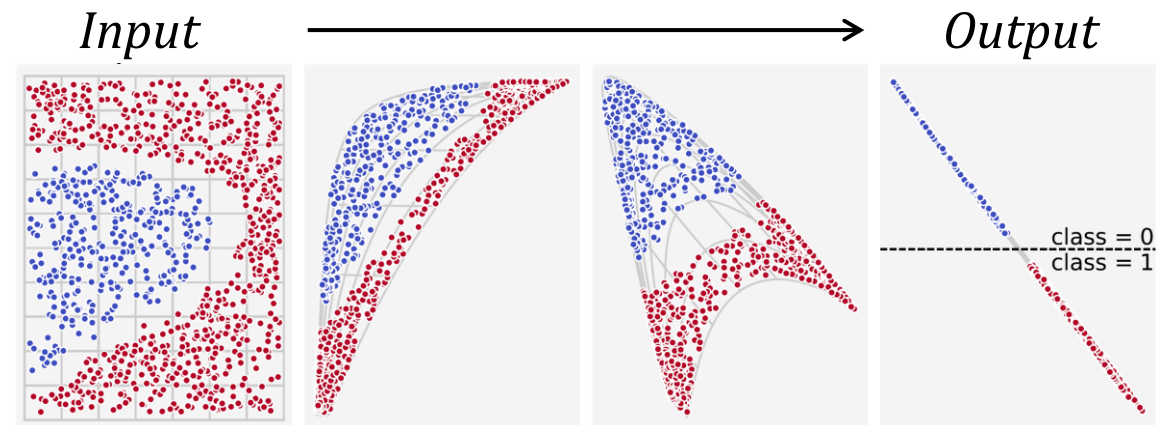
Inductive Biases: model

Example: type of model / architecture, f_θ

Neural network (multilayer perceptron):

- Apply a sequence of linear transformations + point-wise nonlinearities
 - $\mathbf{h}_l = \sigma(\mathbf{W}_l \mathbf{h}_{l-1} + \mathbf{b}_l)$
- Repeat for all layers

(Parameters θ usually consider $\mathbf{W}_*$ and $\mathbf{b}_*$)

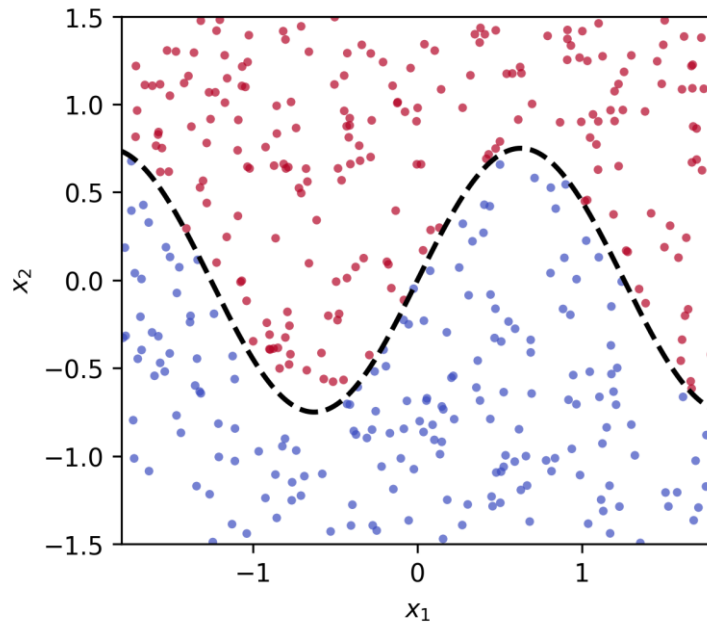


- We progressively warp from input to output space
 - If activation σ is smooth this is a smooth transformation!

Inductive Biases: boundaries

Classify $\mathbf{x} = (x_1, x_2)$ into one of two classes $y \in \{0, 1\}$ (red/blue)

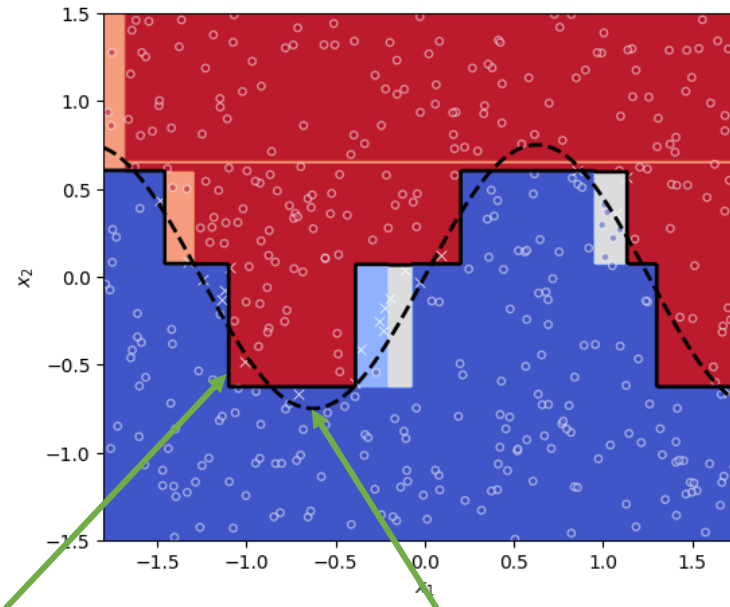
Training data



(sklearn)

DecisionTreeClassifier

max depth = 8, min samples per leaf = 4



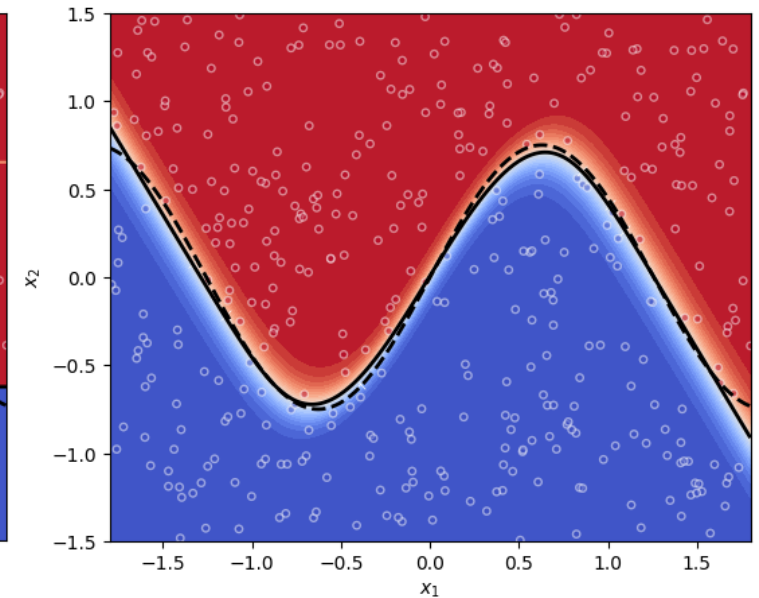
Model decision boundary

Real decision boundary

(torch)

Neural Network

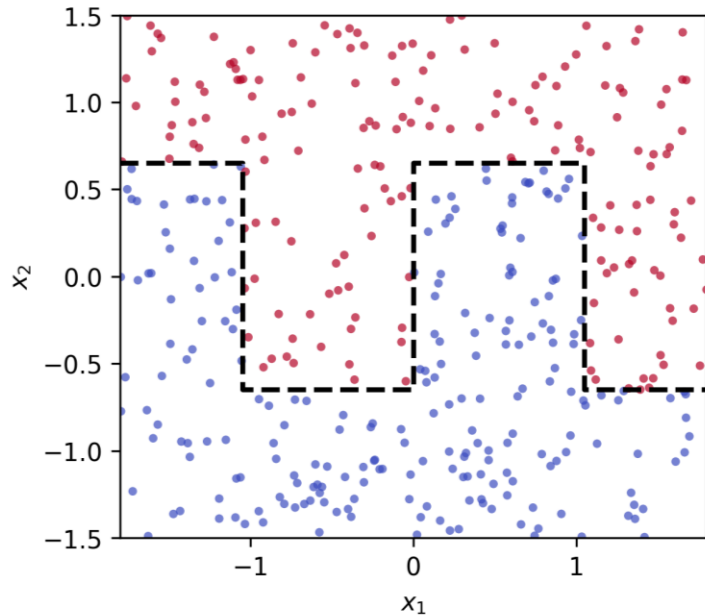
2 hidden layers, 8 hidden dimensions, tanh activation



Inductive Biases: boundaries

Other way around!

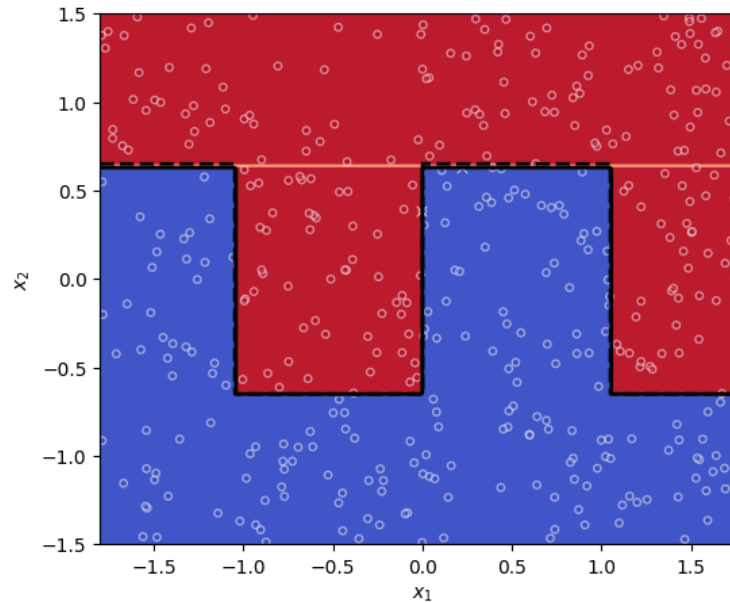
Training data



(sklearn)

DecisionTreeClassifier

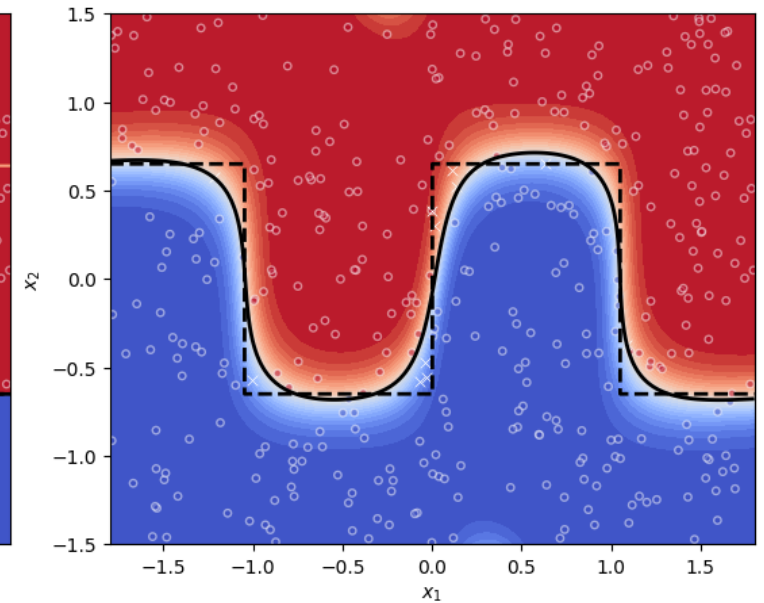
max depth = 8, min samples per leaf = 4



(torch)

Neural Network

2 hidden layers, 8 hidden dimensions, tanh activation

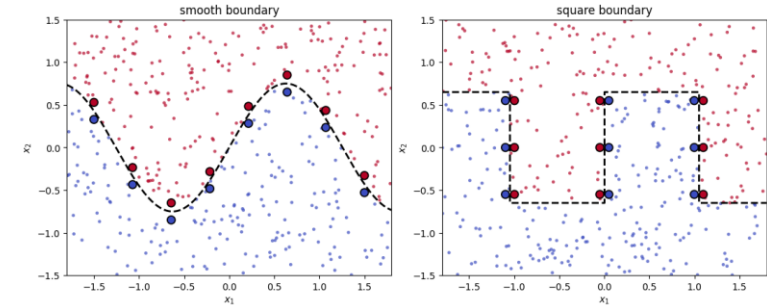


Inductive Biases: boundaries

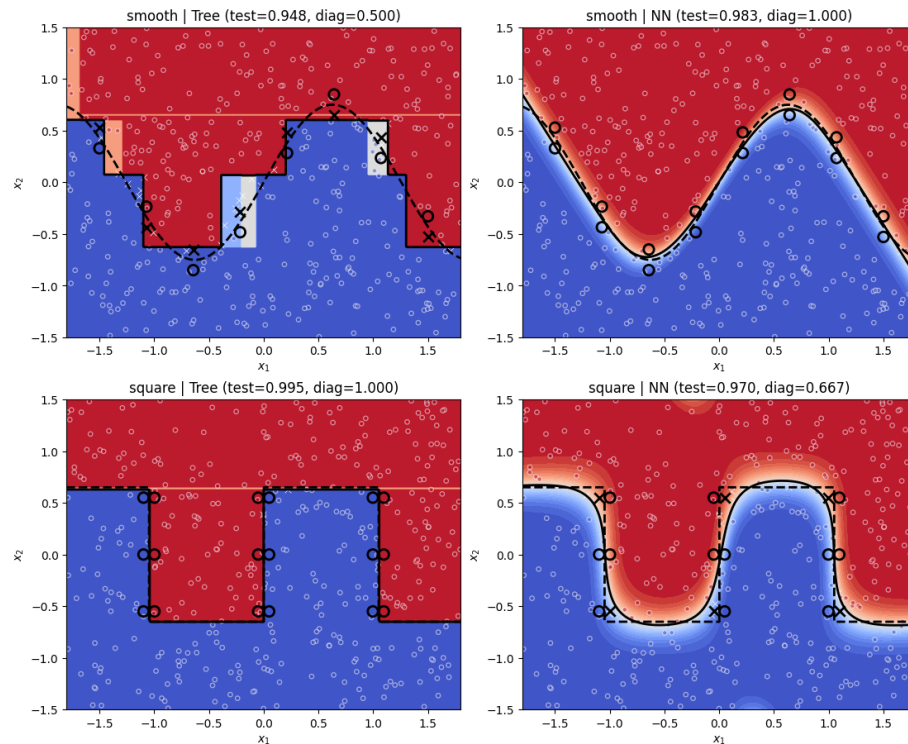
Hands-on intermission:

synthetic/inductive_bias_boundary.ipynb

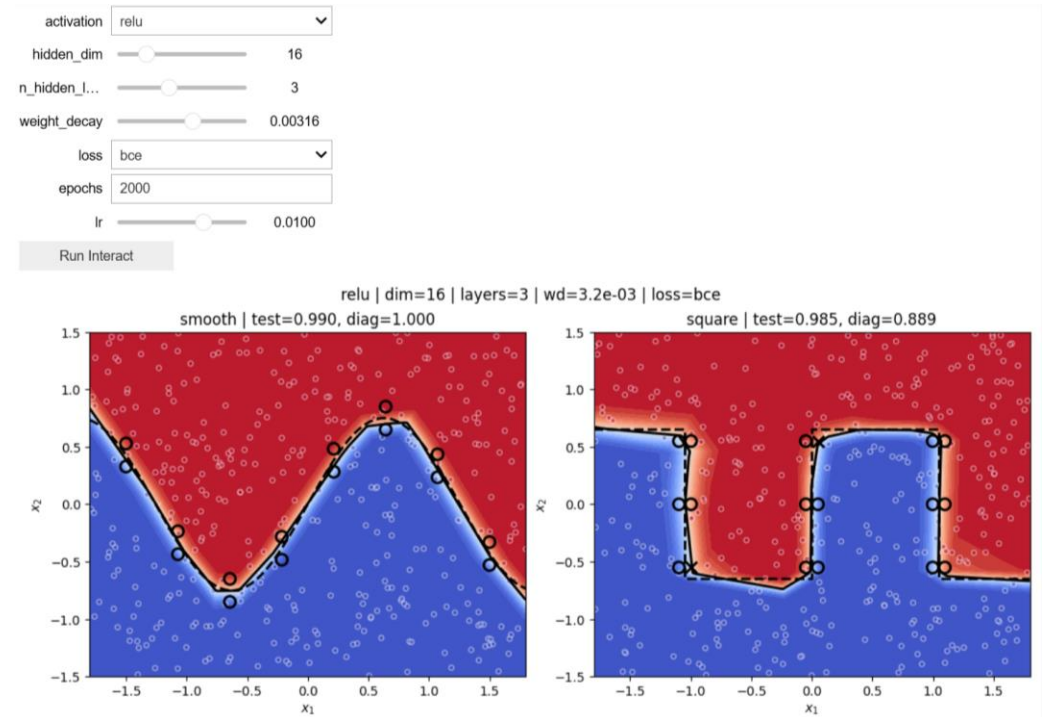
Generate data + diagnostic points near boundary



(A) Decision Tree vs NN



(B) NN hyperparameter playground



Inductive Biases: patterns

Another view: what types of patterns can we*/do we expect to capture?

*Many model classes are universal approximators and can fit anything, but we want to generalize beyond the training data

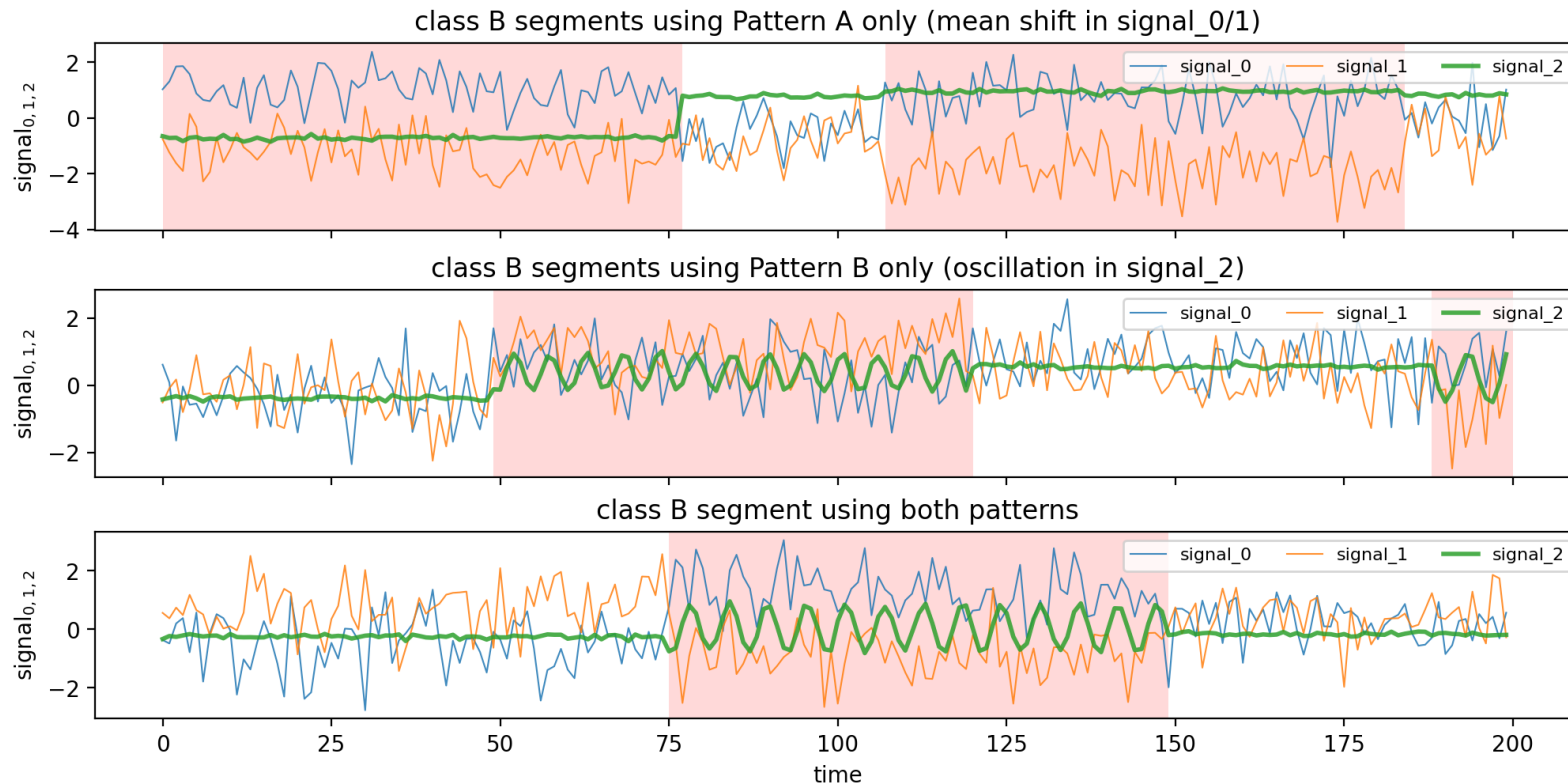
Given some input signals (e.g. measurements over time)

- A. How do we represent our input->output map?
- B. How do we formulate the model (learning)?

Inductive Biases: patterns

Example: per-timestep binary classification of time-series data

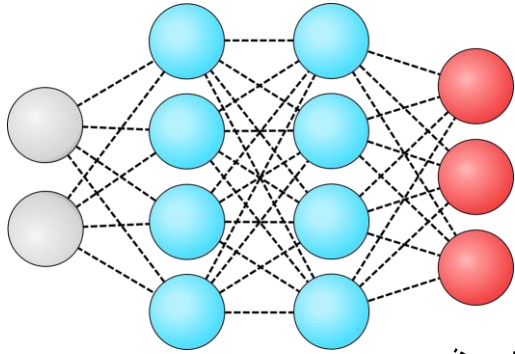
- Three input time-series signals (0, 1, 2) with per-segment mean + gaussian noise
- Distinct parameters for class A/B, where B is defined by two distinct patterns:
 - [Pattern A] Shift of mean in signal 0/1 to distinct range (present in ~80% of samples)
 - [Pattern B] Rapid oscillations in signal 2 (present in ~90% of samples)



Inductive Biases: patterns

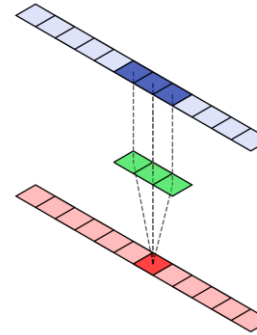
Fully-connected NN

absolute dependence on each input

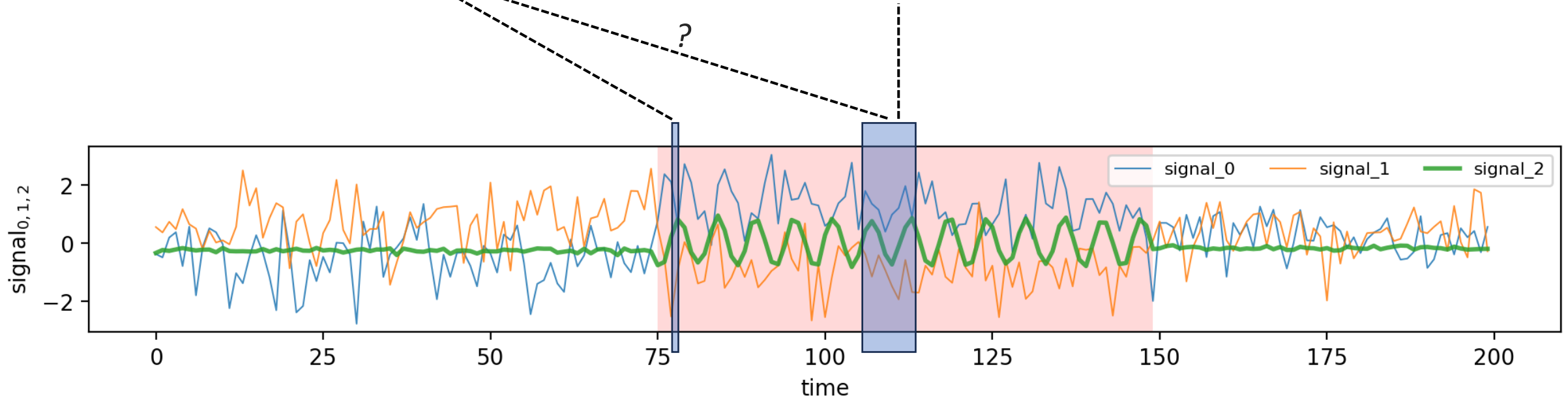


Convolutional NN

learn filters that slide over an axis



[Pattern A] mean shift
[Pattern B] rapid oscillations



Inductive Biases: patterns

Fully-connected NN

absolute dependence on each input

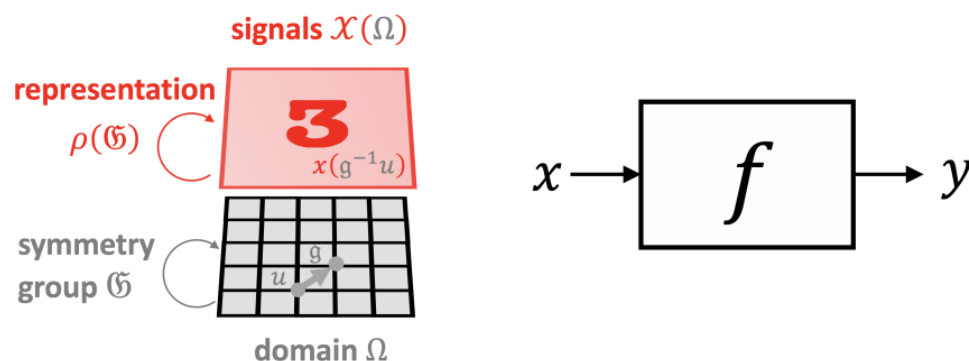


Convolutional NN

learn filters that slide over an axis

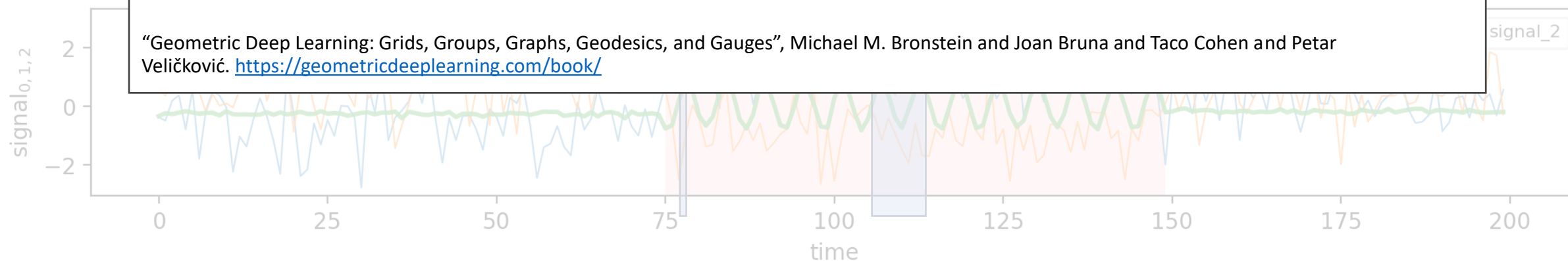


[Pattern A] mean shift
[Pattern B] rapid oscillations



Architecture	Domain Ω	Symmetry group $\mathcal{G}$
CNN	Grid	Translation
Spherical CNN	Sphere / $SO(3)$	Rotation $SO(3)$
Intrinsic / Mesh CNN	Manifold	Isometry $\text{Iso}(\Omega)$ / Gauge symmetry $SO(2)$
GNN	Graph	Permutation Σ_n
Deep Sets	Set	Permutation Σ_n
Transformer	Complete Graph	Permutation Σ_n
LSTM	1D Grid	Time warping

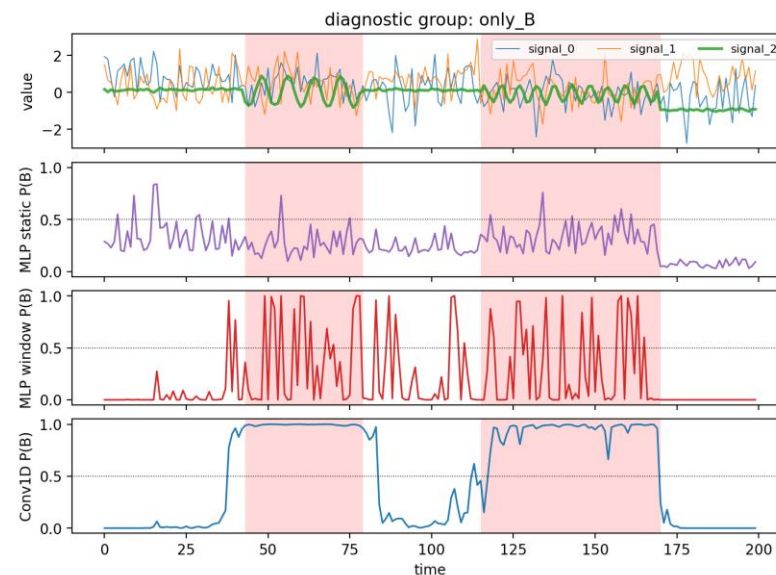
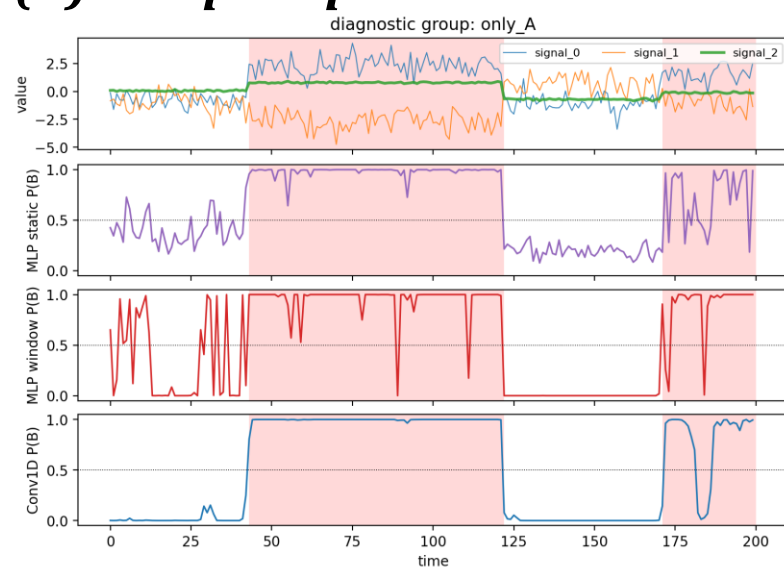
“Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges”, Michael M. Bronstein and Joan Bruna and Taco Cohen and Petar Veličković. <https://geometricdeeplearning.com/book/>



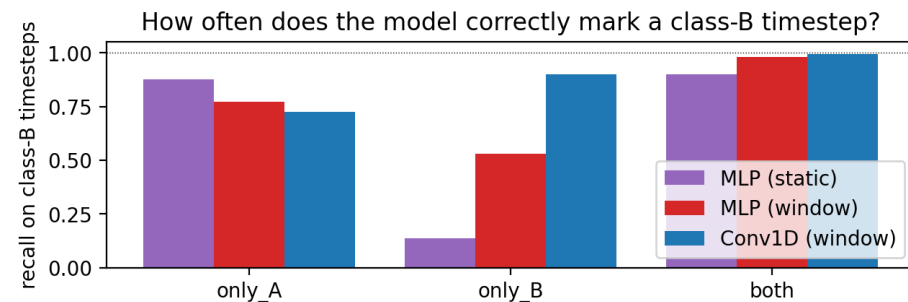
Inductive Biases: patterns

Let's compare: `synthetic/inductive_bias_conv.ipynb`

(A) Compare prediction traces



(B) Per-pattern metrics



Ensembling: let's combine!

$$\theta^* = \arg \min_{\theta \in \Theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\theta}(x_i), y_i)$$

Utilize the diversity by building ensembles

- We can train models with different formulations, optimization runs (or strategies), objectives or even using different data (splits)
- Use the combined predictions, (dis-)agreement to:
 - Improve performance (“wisdom of the crowds”)
 - Trust predictions more/less (quantify the uncertainty/confidence)

Ensembling: let's combine!

First, train a set of M diverse models $f_1, f_2, \dots, f_M$ independently

For inference, produce predictions $\tilde{y}_1, \tilde{y}_2, \dots, \tilde{y}_M$ and combine with various possible approaches:

- Simple average; $\tilde{y} = \frac{1}{M} \sum_{i=1}^M \tilde{y}_i$
- Weighted average; $\tilde{y} = \sum_{i=1}^M w_i \tilde{y}_i$
- Mixture of experts (adaptive weights); $\tilde{y} = \sum_{i=1}^M w_i(x) \tilde{y}_i$
- Stacking (meta-model that combines); $\tilde{y} = g_{\theta}(\tilde{y}_1, \tilde{y}_2, \dots, \tilde{y}_M)$
- (Classification) majority voting, i.e., of $\tilde{c}_i = \arg \min_c \tilde{y}_{i,c}$
- ...

Ensembling

Focus on improving prediction $\tilde{y}$

- Accuracy
- $\tilde{y}$ represents class probabilities $\rightarrow$ how well do they represent real probabilities?

*Probability/confidence **calibration***

Ensembling

Focus on improving prediction $\tilde{y}$

- Accuracy
- $\tilde{y}$ represents class probabilities → how well do they represent real probabilities?

*Probability/confidence **calibration***

Could also use to quantify uncertainty through disagreement, e.g.:

- Model uncertainty as variance (epistemic): $\sigma_{model}^2 = \frac{1}{M} \sum_{i=1}^M (\tilde{y}_i - \bar{y})^2$
- If models give data uncertainty (aleatoric), average: $\sigma_{data}^2 = \frac{1}{M} \sum_{i=1}^M \sigma_{i,data}^2$

Closely related concepts but not quite the same!

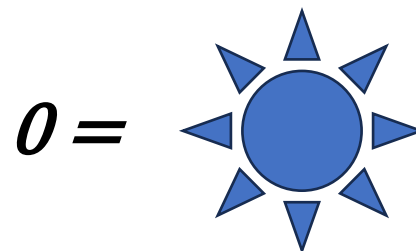
Calibration

Assume binary classification $\rightarrow$ the model output $\tilde{y} \in [0, 1]$ represents the probability of a positive (> 0.5) or negative (< 0.5) outcome

- Accuracy only covers how often we are on the “right side” of 0.5
- Calibration concerns whether the probability itself is meaningful
 - Do predictions with $\tilde{y} = 0.95$ resolve to positive about 95% of the time? Predictions with $\tilde{y} = 0.60$ about 60%? etc.

Calibration: example

Is it going to rain tomorrow?



No!

$$p(\text{rain}) = \tilde{y}_A = 0.45$$



B



Yes!

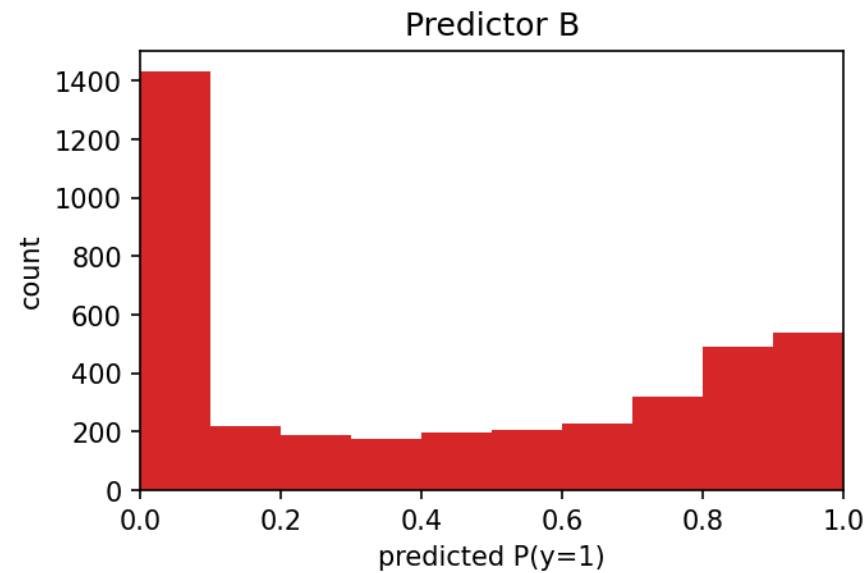
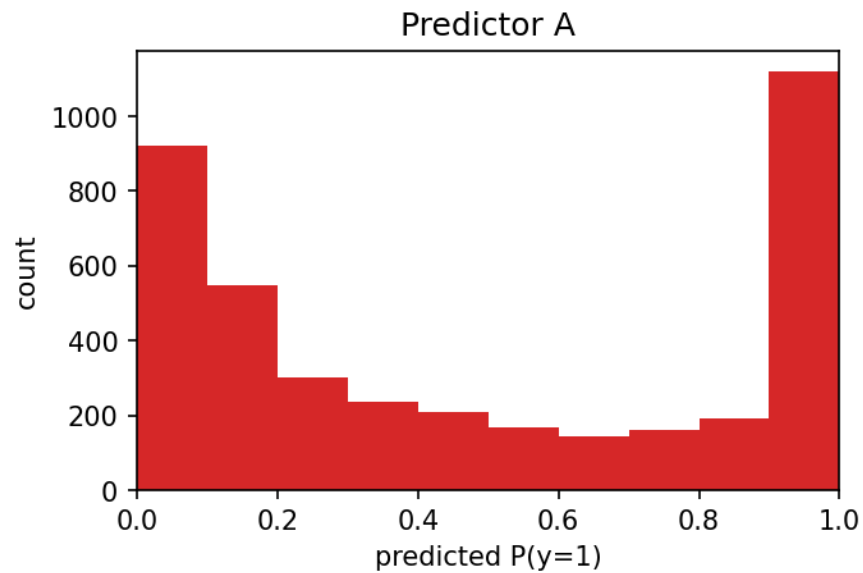
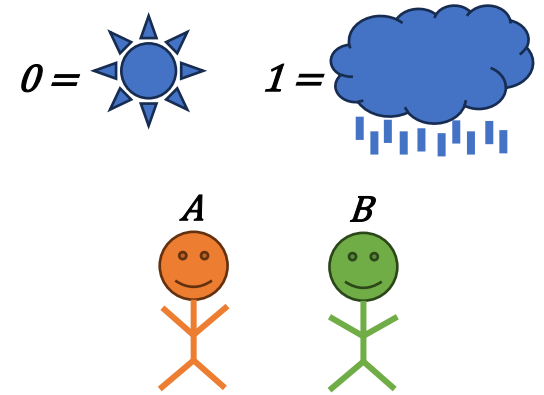
$$p(\text{rain}) = \tilde{y}_B = 0.7$$

$$\text{accuracy} = \frac{1}{N} \sum_{i=1}^N 1((\tilde{y}_{A/B,i} \geq 0.5) = y_i) =$$

$\tilde{y}_B$	$\tilde{y}_B$	y
0.75	0.65	1
0.9	0.55	1
0.6	0.45	0
0.55	0.8	1
0.3	0.55	0
77.4%	77.2%	

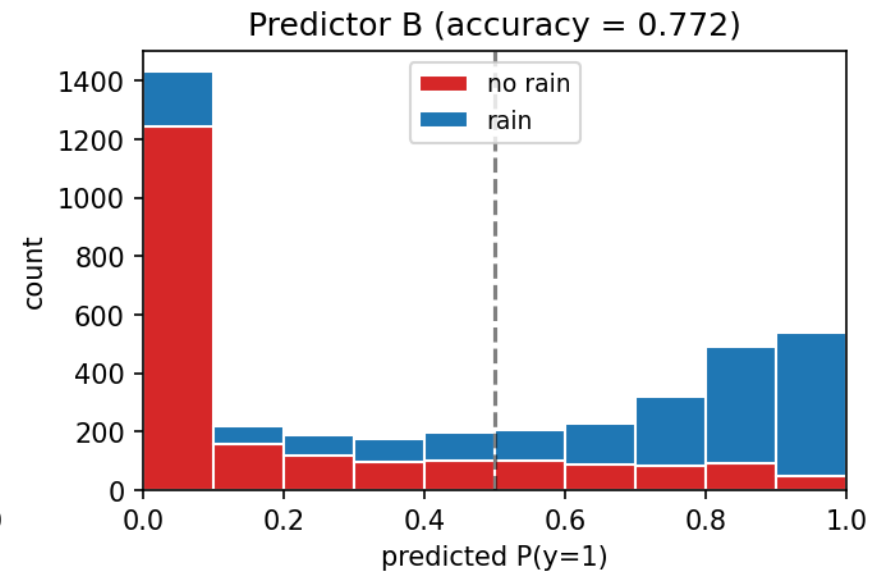
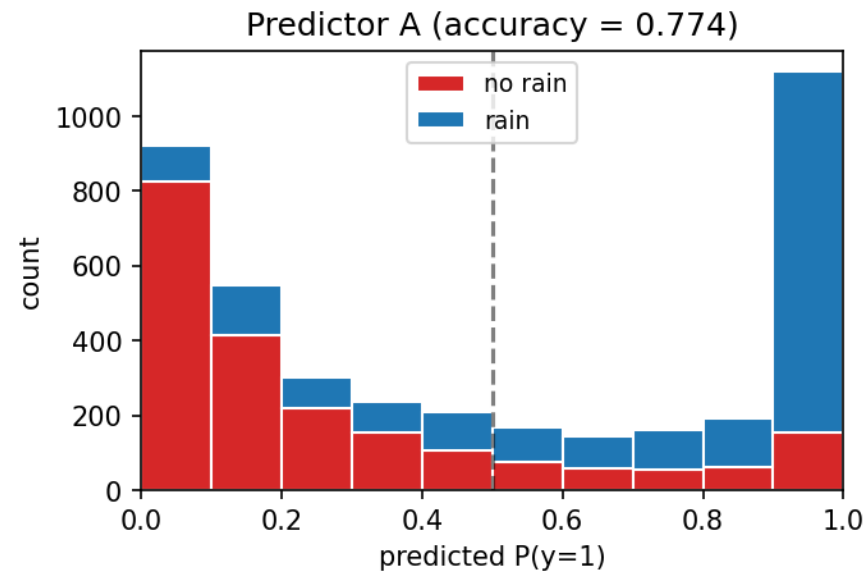
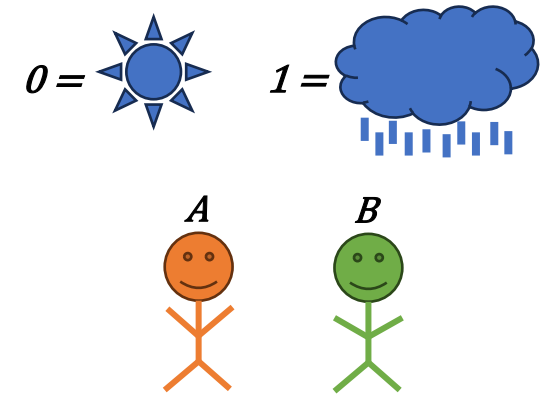
Calibration: example

How do they predict on the aggregate?



Calibration: example

How do they predict on the aggregate?



→ If well calibrated, overwhelmingly red near 0, blue near 1, 50:50 in the middle

Calibration: metric

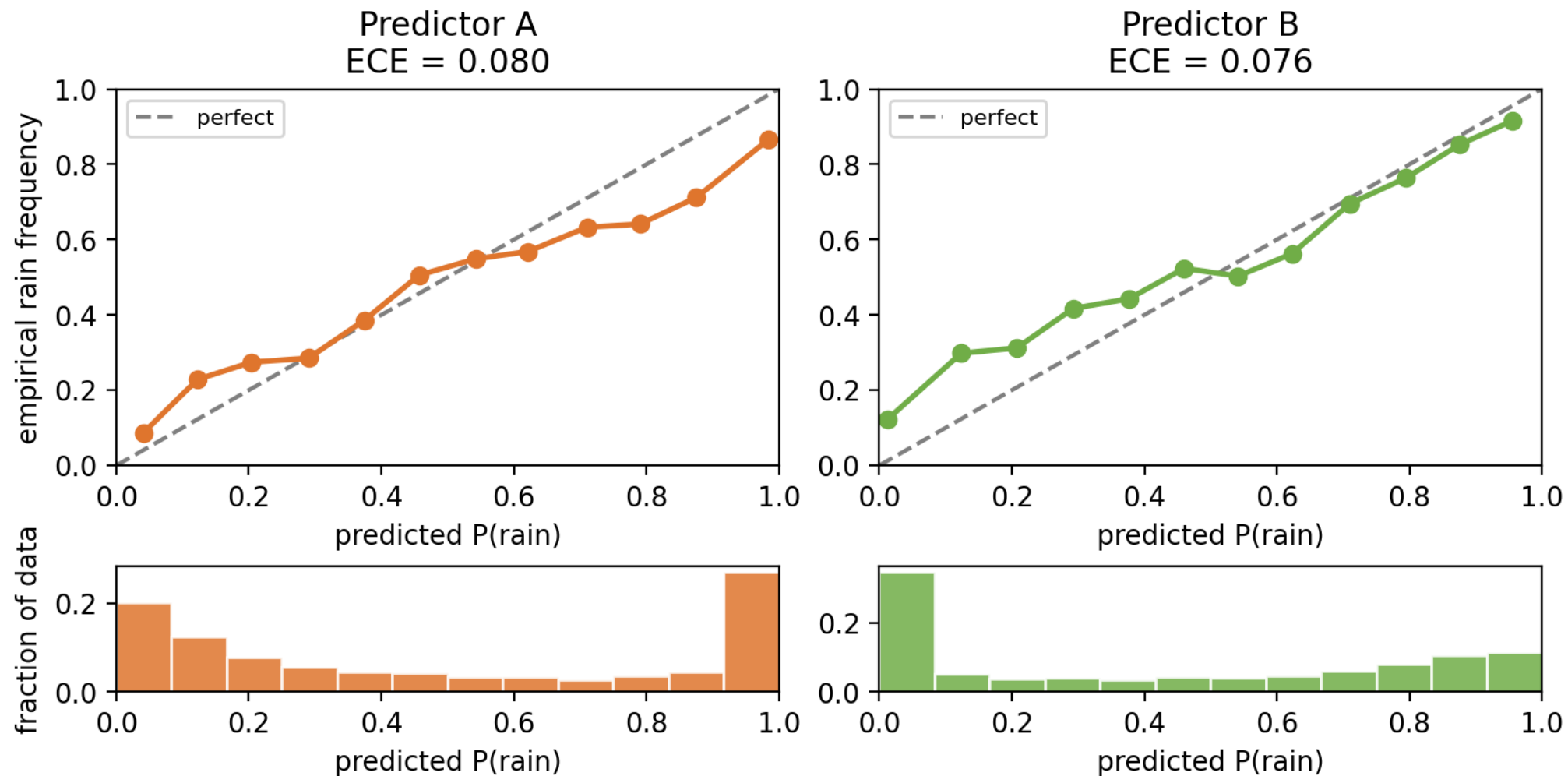
To quantify how well the predictions represent the real probabilities, discretize along the probability axis to compute the Expected Calibration Error (ECE):

$$ECE = \sum_{b=1}^B \frac{len(b)}{N} |acc(b) - conf(b)|$$

- Bin predictions by confidence (e.g. B bins of width 0.1: $[0, 0.1]$, $(0.1, 0.2]$, ..., $(0.9, 1.0]$)
- Per bin b , compute the accuracy $acc(b)$ (fraction of correct predictions), the confidence $conf(b)$ (average predicted probability) and the bin size $len(b)$.
- Compute the absolute difference per bin, weight by bin size and sum.

Calibration: metric

To visualize, plot bin confidence vs. accuracy: reliability diagrams



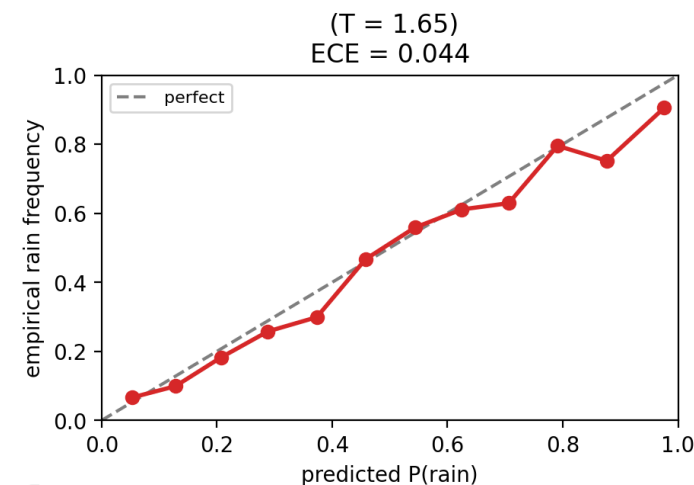
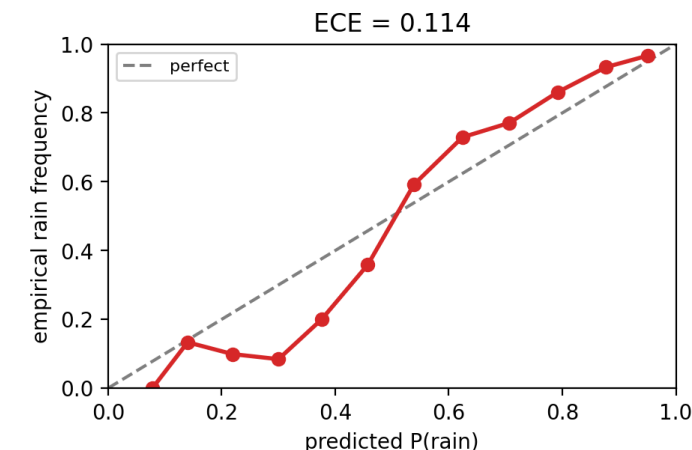
Calibration: post-hoc fixes

Sometimes, model captures dependency between confidence and accuracy, but poorly scaled (often models are not trained with calibration explicitly in mind, e.g. cross-entropy pushing NNs towards extremes)

→ Apply post-hoc scaling using some holdout data

For example, temperature scaling: divide logit z ('raw' prediction) by scalar T and renormalize, e.g.

- Binary: $y_T = \text{sigmoid}(z/T)$
- Multi-class: $y_T = \text{softmax}(z/T)$

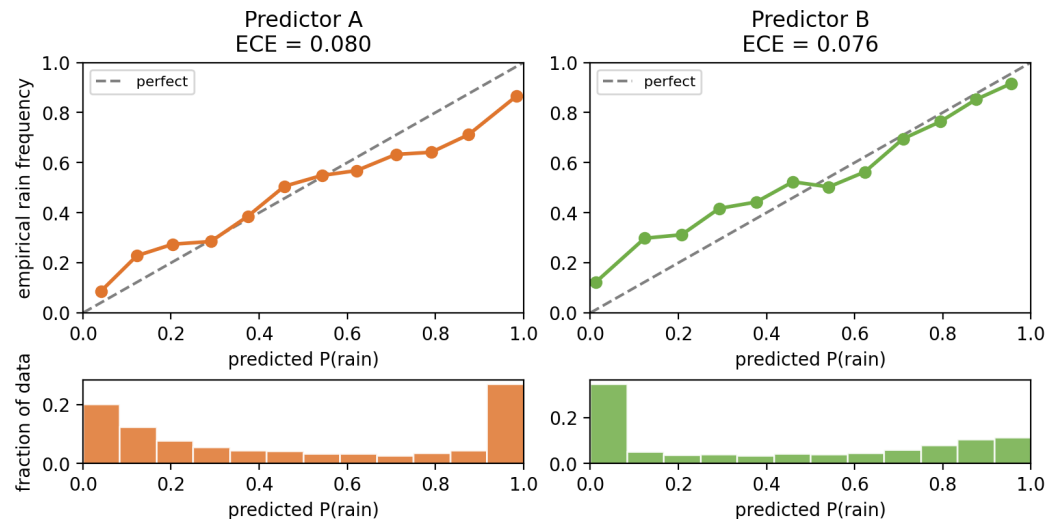


Calibration: demo

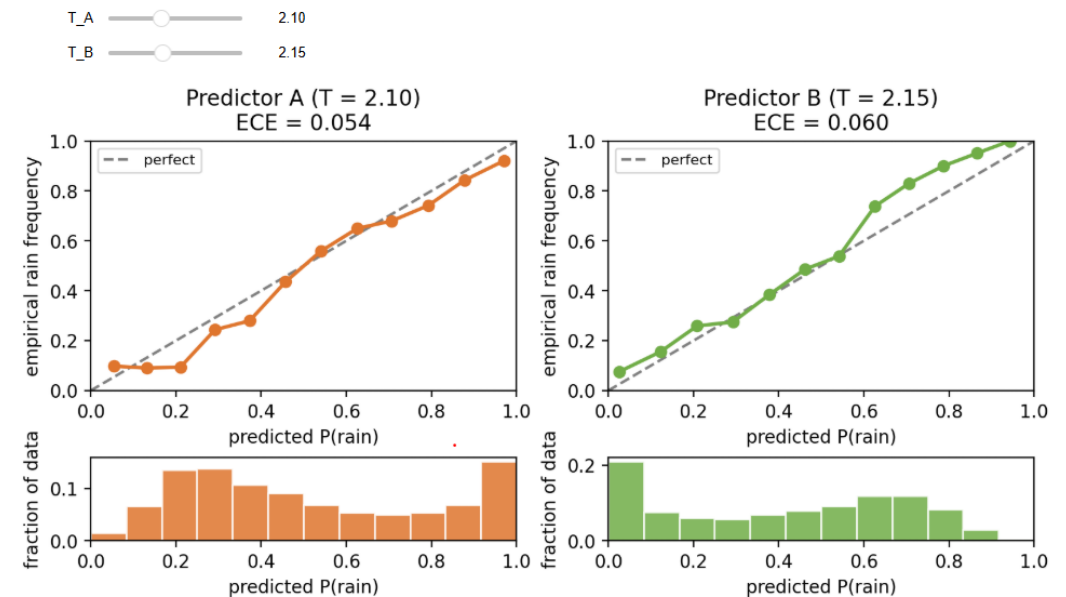
Calibration and ensembling demo: synthetic/calibration_ensembling.ipynb

(A) Measuring calibration

$$\text{ECE} = \sum_{b=1}^B \frac{n_b}{N} |\bar{p}_b - \bar{y}_b|$$



(B) Temperature scaling +interactive playground



(C) Ensembling the predictors

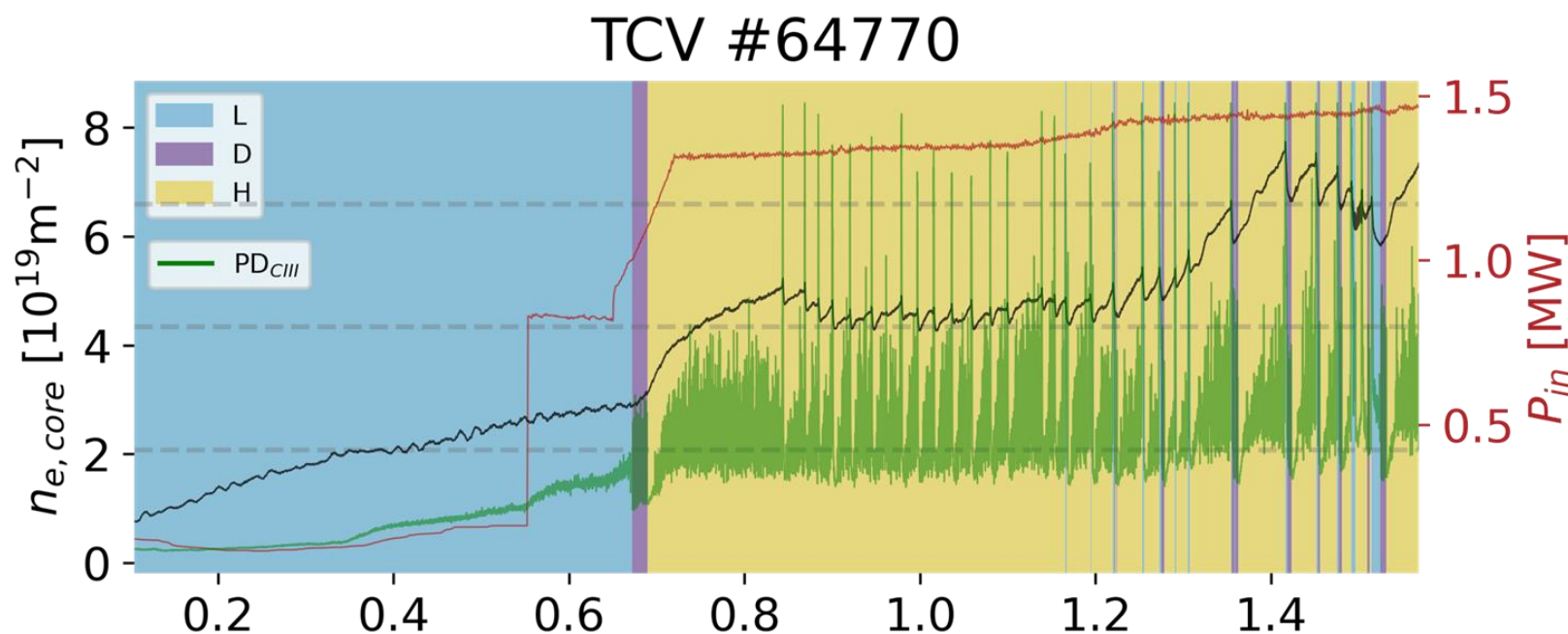
$$p_{\text{ens}} = \frac{1}{2} p_A + \frac{1}{2} p_B.$$
$$p_{\text{conf}} = \frac{w_A p_A + w_B p_B}{w_A + w_B}.$$

Real-world example

Classify a time series of TCV shot measurements into different confinement regimes, for L-mode/dithering/H-mode:

$$f : \mathbf{x}^{\mathbf{u}, \mathbf{t}^{in}} \in \mathbb{R}^{U \times T^{in}} \rightarrow \mathbf{y}^{\mathbf{s}, \mathbf{t}^{out}} \in \mathbb{R}^{3 \times T^{out}} \quad \mathbf{s} = \{L, D, H\}$$

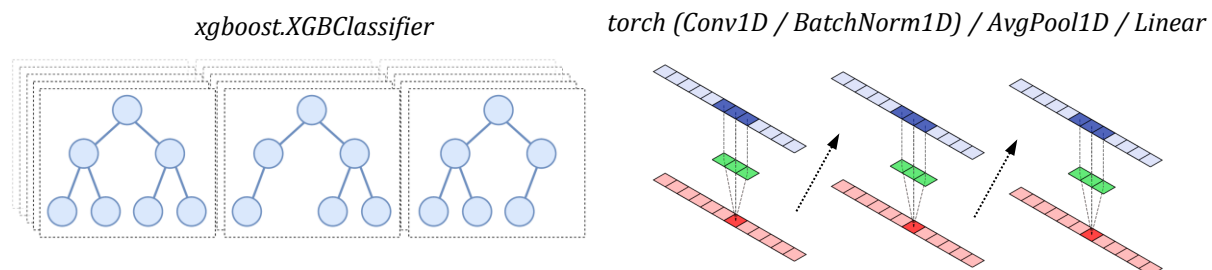
no. input signals no. input timesteps no. output timesteps



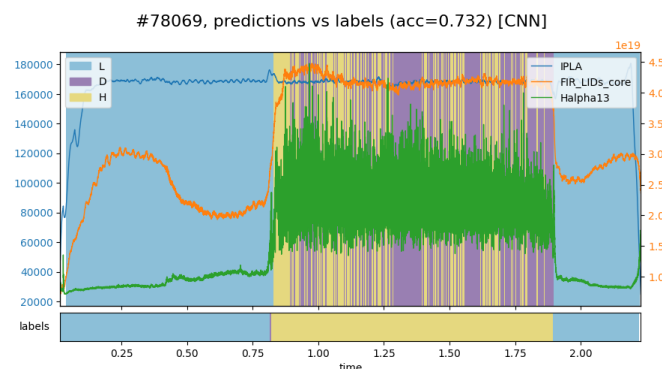
Real-world example*

TCV/tcv_conf_state.ipynb (download data: TCV/download_dataset.ipynb, subset of TCV confinement state dataset: [zenodo](#))

(A) Train Conv1D and XGBoost classifiers



(B) Compare metrics, regime-specific failure modes

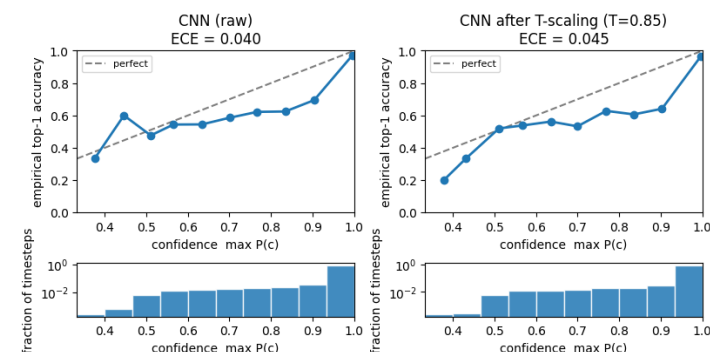


→ Overfit to ELMs, fail in QCE

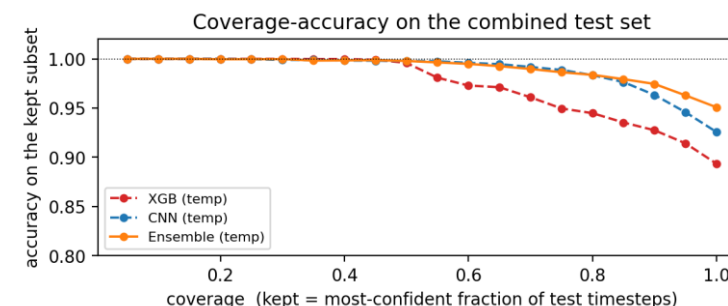
	group	n	XGB	CNN	Ensemble
0	small-ELM	3098	0.983	0.810	0.984
1	other	19424	0.879	0.944	0.945
2	all test	22522	0.894	0.926	0.951

→ CNN strong otherwise, ensembling for best of both worlds

(C) ECE, temperature scaling



(D) Calibrate → ensemble, coverage-accuracy



*of course, the example is a bit idealized / tuned to the tutorial :) In practice, it is problem dependent. Also, data is everything! If we have enough samples, we can learn many things from data (see also, power of transformers when data size goes ↑↑)